

Logic Building & Java Foundations

The Builder's Journey

```
public class Architect {  
    void buildLogic(Code base) {  
        System.out.println('Foundation laid.');    }  
  
    // Transitioning blueprint to bytecode  
    Architect student = new Architect();  
    student.execute(Plan.JAVA);  
    student.execute(Plan.JAVA);  
}
```



The Mindset

(Logic & Habits)

Shaping the problem-solving engine.



The Blueprint

(Algorithms & Ecosystem)

Designing the flow and understanding Java's architecture.



The First Bricks

(Syntax & OOP)

Grounding theory into functional code.

The Logic Gym: Lifting Weights for Your Brain

Software Building (Making Clones)

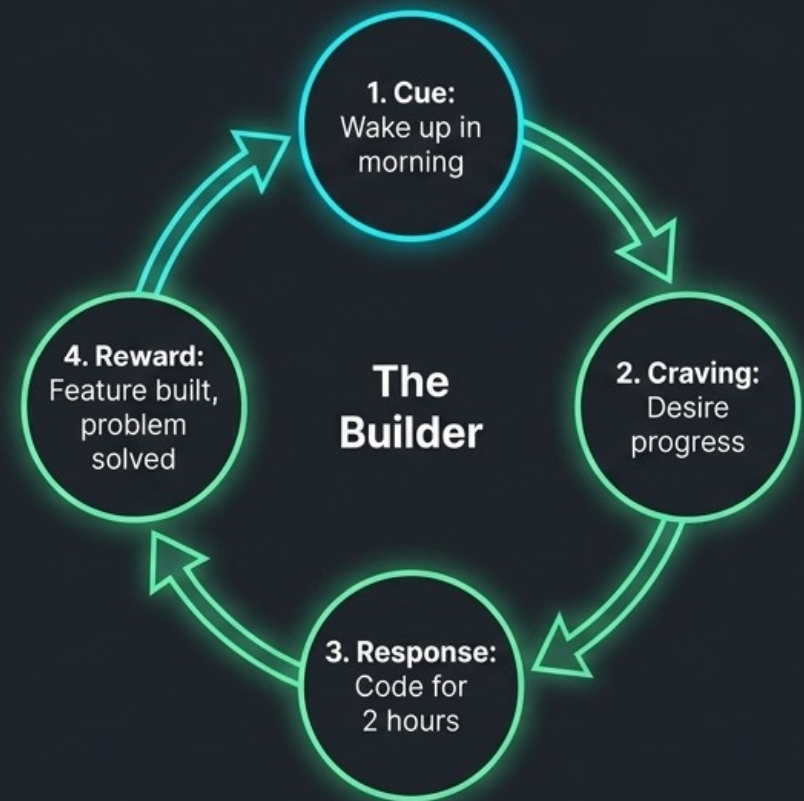


Problem Solving (Understanding Core Tech)



You can't get fit just by watching a bodybuilder; you have to lift the weights. Programming syntax is the machine. Logic building is the muscle.

The Developer's Habit Loop

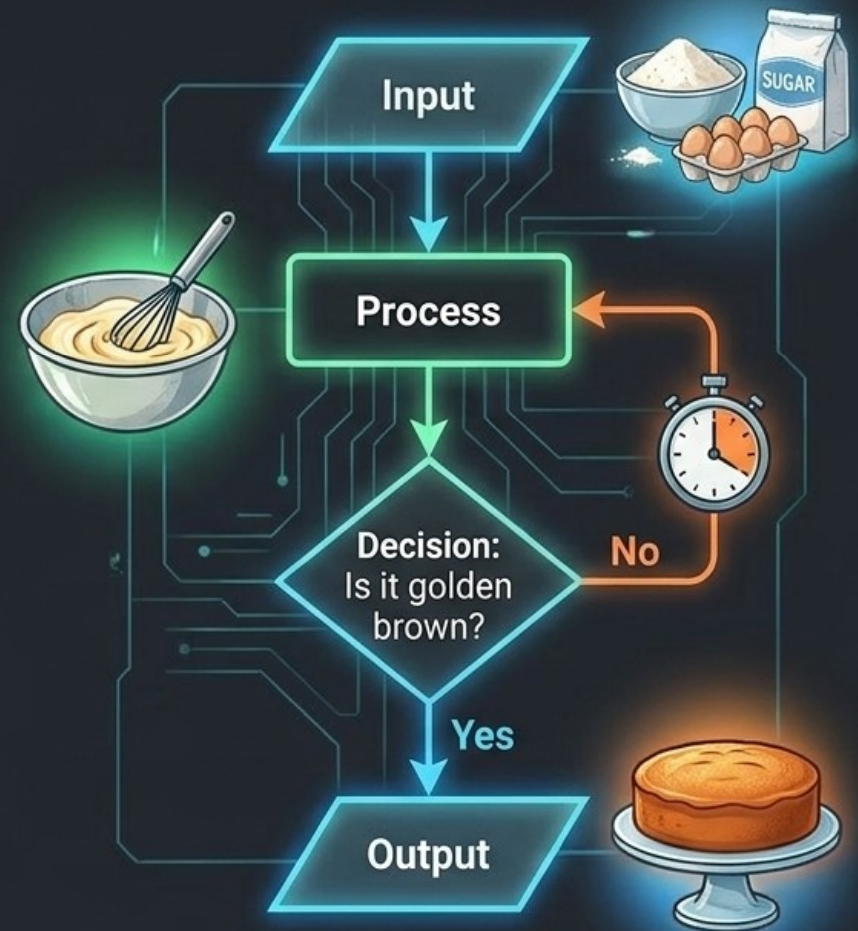


Consistency beats intensity. Build small features daily.

Algorithms: The Ultimate Blueprint

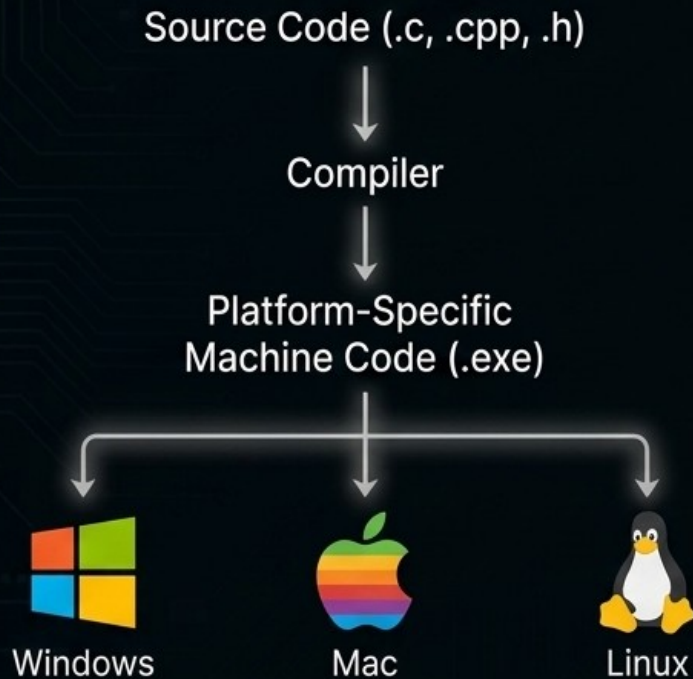
Clear, unambiguous, step-by-step instructions with a defined input and output.

Like a recipe for baking a cake:
If you miss a step or change the order, the output fails.



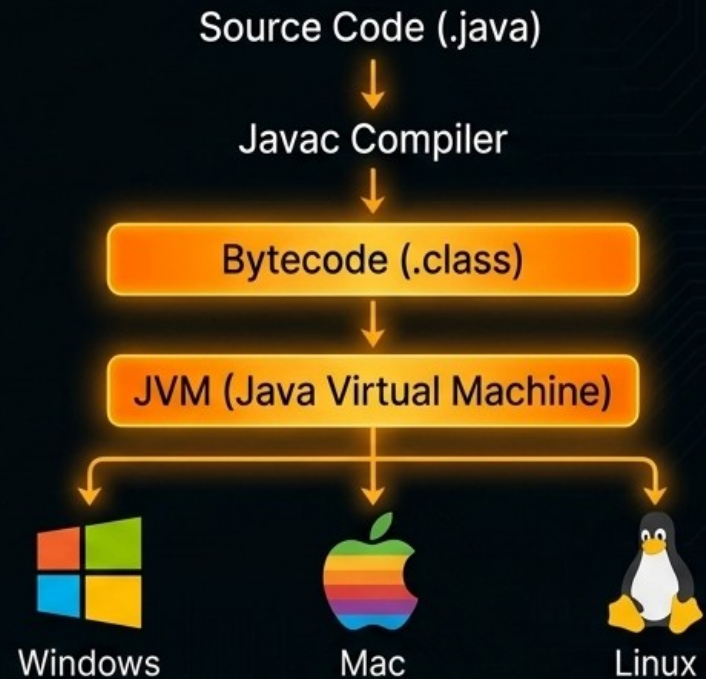
Why Java? The WORA Revolution

C/C++ (1970s)



Platform Dependent

Java (1995)



Write Once, Run Anywhere (WORA)

Demystifying the Ecosystem

JDK (Java Development Kit)

JRE (Java Runtime Environment)

The Environment to play in:

Contains JVM + Java Class Libraries

JVM (Java Virtual Machine)

The Engine running the code:

Heap Memory

Stack Memory

PC Register

JIT Compiler

The Construction Kit:

Contains JRE + Development
Tools (compilers, debuggers)

JDK

Tools For developing applications

JRE

JVM

Heap Memory

Stack Memory

Memory area (non-Heap)

PC Register

⋮

JIT

Java class Libraries
(java.lang, java.io,
java.util, etc)

Java standard
extensions
(JavaFX, JCE, etc)

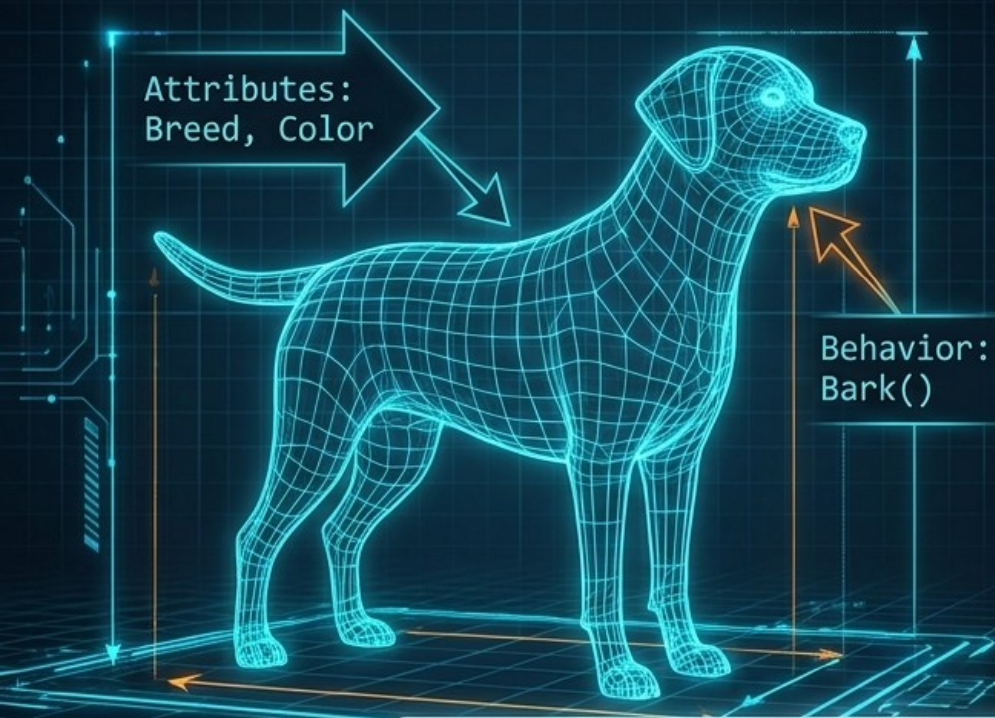
Key Features of Java



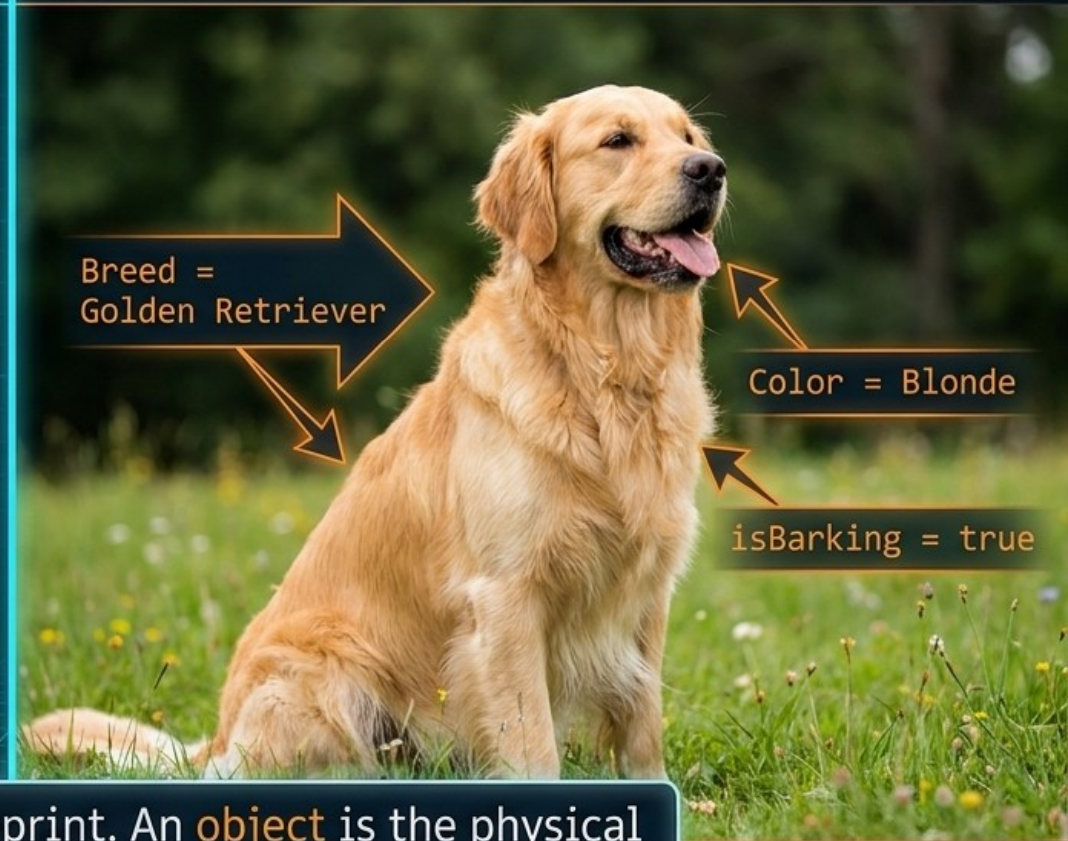
Garbage Collection acts as an automated background janitor, quietly cleaning up unused memory so the developer doesn't have to.

OOP Core: Classes vs. Objects

Class (The Template)

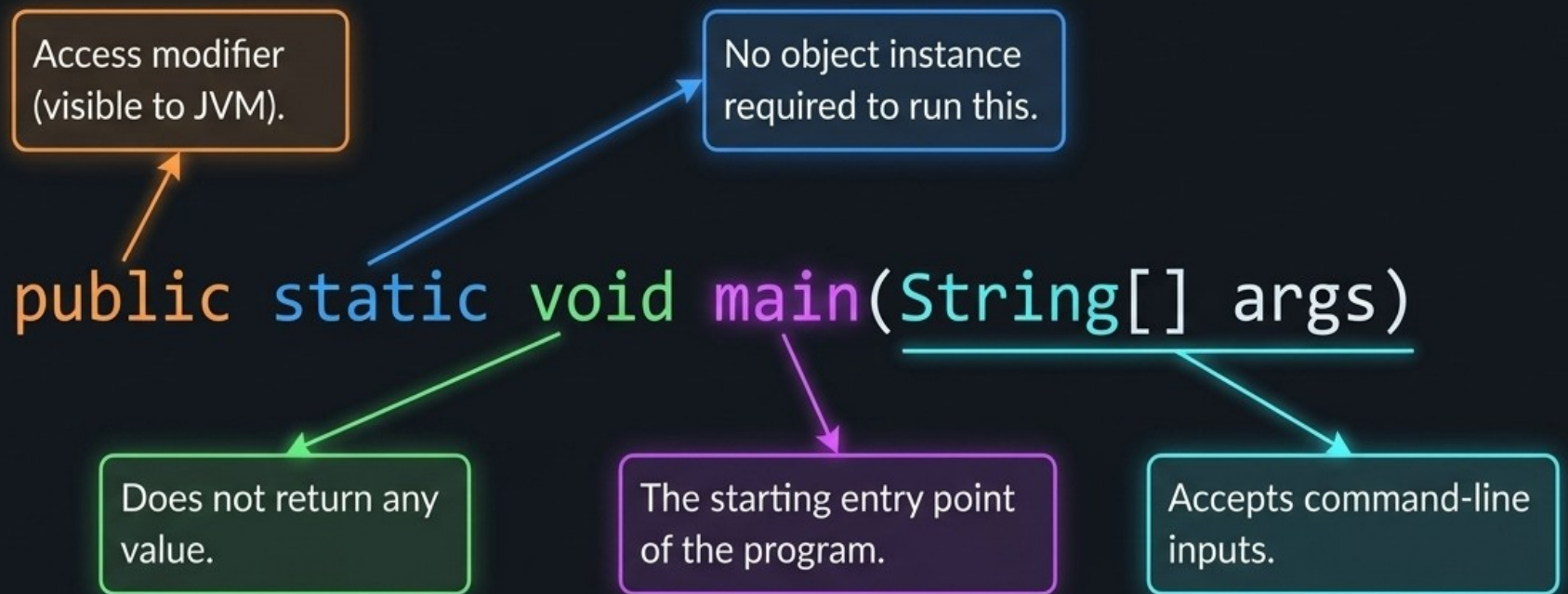


Object (The Instance)



A **class** is the conceptual blueprint. An **object** is the physical realization of that blueprint in the system's memory.

Anatomy of Hello World



The Execution Pipeline



Program.java
(Human-readable Source)



Javac
(Java Compiler)



Program.class
(Bytecode)



JVM
(Universal Translator)



Display
(**'Hello World'** Output)

The Ancestry: Root Classes



The Java Lexicon: Reserved Keywords

Data Types

boolean

byte

char

short

short

Modifiers

abstract

final

private

protected

protected

public

static

Control Flow

break

case

continue

default

do

else

switch

while

Exceptions

catch

finally

throw

throws

try

Interview Trap: True, False, and Null are literals, NOT keywords. They cannot be used as identifiers.

Java Keywords

Method	Description
abstract	A non-access modifier.
assert	For debugging
boolean	A data type that can only store true or false values
break	Breaks out of a loop or a switch block
byte	A data type that can store whole numbers from -128 and 127
case	Marks a block of code in switch statements
catch	Catches exceptions generated by try statements
char	A data type that is used to store a single character
class	Defines a class
const	Defines a constant. Not in use - use <u>final</u> instead
continue	Continues to the next iteration of a loop
default	Specifies the default block of code in a switch statement
do	Used together with while to create a do-while loop

Java Keywords

Method	Description
else	Used in conditional statements
enum	Declares an enumerated (unchangeable) type
extends	Extends a class (indicates that a class is inherited from another class)
final	A non-access modifier used for classes, attributes and methods, non-changeable
finally	Used with exceptions, a block of code that will be executed no matter of an exception or not
float	A data type that can store fractional numbers from 3.4e-038 to 3.4e+038
for	Create a for loop
goto	Not in use, and has no function
if	Makes a conditional statement
implements	Implements an interface
import	Used to import a package, class or interface
instanceof	Checks whether an object is an instance of a specific class or an interface
int	A data type that can store whole numbers from -2147483648 to 2147483647

Java Keywords

Method	Description
long	Store whole numbers from -9223372036854775808 to 9223372036854775808
native	Specifies that a method is not implemented in the same Java source file (but another language)
new	Creates new objects
package	Declares a package
private	An access modifier used for attributes, methods and constructors, making them only accessible within the declared class
protected	An access modifier used for attributes, methods and constructors - accessible in the same package and subclasses
public	An access modifier used for classes, attributes, methods and constructors, making them accessible by any other class
return	Finished the execution of a method, and can be used to return a value from a method
short	A data type that can store whole numbers from -32768 to 32767
static	A non-access modifier. Static methods/attributes can be accessed without creating an object of a class
strictfp	Obsolete. Restrict the precision and rounding of floating point calculations
super	Refers to superclass (parent) objects
switch	Selects one of many code blocks to be executed
synchronized	A non-access modifier, which specifies that methods can only be accessed by one thread at a time

Java Keywords

Method	Description
this	Refers to the current object in a method or constructor
throw	Creates a custom error
throws	Indicates what exceptions may be thrown by a method
transient	Used to ignore an attribute when serializing an object
try	Creates a try...catch statement
void	Specifies that a method should not have a return value
volatile	Indicates that an attribute is not cached thread-locally, and is always read from the "main memory"
while	Creates a while loop

True, **false** and **null** are not keywords, but they are literals and reserved words that cannot be used as identifiers.

The Builder's Mandate

1. **Mindset:** Embrace the cognitive workout. Logic is a muscle.
2. **Blueprint:** Understand the architecture (WORA, JVM) before writing the code.
3. **Bricks:** Master the foundational OOP concepts and core lexicon.

Do a dry run in your mind. Write code on pen and paper first. Build small. Structure and consistency yield success.

